CONFIDENTIAL: FOR INTERNAL USE ONLY

Behavior Trees Framework for ROS

Revision History

Revision	Date	Author(s)	Description
0.1	22.01.19	Álvaro Páez	created

List of Figures

List of Tables

List of Code Snippets

Contents

1 Overview

Behavior Trees (BT) are a way to structure the flow and control of multiple tasks in the context of decision-making applications. They are usually presented as an alternative to Finite State Machines (FSM). First originated as a tool for NPC AI development in the video game industry, BT have become widespread in robotics, mainly due to its modularity and simplicity [1].

This document aims to serve both as a technical description and as user documentation on how to make use of behavior trees in a ROS environment. First, a basic introduction to BT is presented, mainly to establish terms and conventions that will be used throughout the document. After that, the general structure and the envisioned work-flow is described. Users looking for a quick reference may focus on the following sections, where a descriptive list of all the ready-to-use nodes is presented. A cookbook is also included, with several how-to's and pitfalls behavior designers may encounter. Finally, annexes include more in-depth descriptions on how stuff work under the hood, and rationale behind some decisions taken during development.

2 Introduction to Behavior Trees

BT model behaviors as hierarchical trees made up of nodes. Trees are traversed from top to bottom at a given rate following a set of well-defined rules and executing the tasks/commands that are encountered while doing so.

Tasks and commands statuses are reported back up in the chain and the flow changes accordingly. A status can be either success, failure or running. It's important to note that these statuses are not associated to a single node, but rather to a parent-child link. This means that parents always get a status coming up through their links, whether what's at the other side it's a single node or a sub-tree.

According to their functionality, nodes can be classified as:

- **Composite**: control the flow through the tree itself and are similar to control structures such as if, while or for loops in traditional programming languages The most common are:
  - Sequence: loops through its children nodes sequentially. It returns success only if all its children return it as well, otherwise returns fail or running. Similar to an AND gate.
  - Selector: sequence's counterpart. It checks each child node in order until one returns success and halts its processing. It can be thought as an OR gate.
- **Decorator**: process or modify the status it receives from its child (it can only have one). An example of this would be the Inverter, that returns the opposite status it receives (e.g. fail in case of success and vice versa).
- **Leaf**: this is where the actual task is performed and commands defined by the user are executed. As such, these nodes cannot have any children.

As it can be seen from the classification above, BT decouple logic from actual tasks in a natural way. When developing a tree one only should care about the leaf nodes. The flow can later be defined and re-arranged constantly, creating new behaviors and expanding on what's already done.

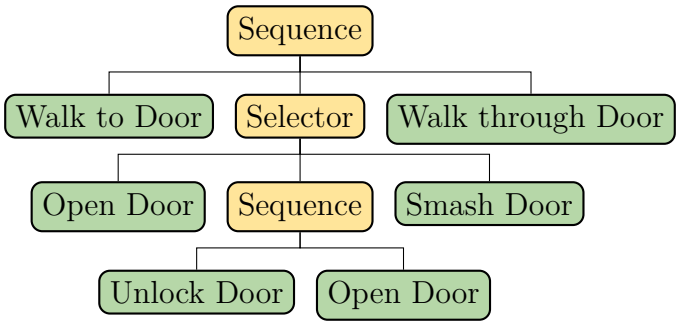


Figure 1: Example of a simple Behavior Tree representing the sequences of actions to open a door.

A simple BT listing the sequence of tasks for a given AI to open a door is depicted in figure 1. Composites are represented as yellow nodes and leaves as green nodes. The well-defined behavior of composites makes it fairly easy to understand how and in what order the different tasks are performed. The root sequence composite ensures, for example,

that the "open door" task will not be started until the "walk to door" task is finished with a success status. On the other hand, selector composites can be seen as a mechanism to list optional ways to undergo a task, going from the most to the least favorable. For example, if it is possible to open the door normally, the first child node will return true and the selector processing will come to halt, returning true to its parent and thus resuming the top sequence (it will not try to smash it).

3 Behavior Trees in ROS

This section shows how to set things up to get trees up and running in your ROS environment. This includes all external dependencies, packages and tools needed to effectively work with them. Several tutorials and examples are included as well. At the end of this section you should be able to create, modify and monitor them at ease.

Trees examples in the following subsections come with default values for topics and other variables. This tutorial assumes you are using the exact same tree with the same values.

3.1 Setting Things Up

There are several libraries and tools that need to be present in the system before you can start working with trees:

- **BehaviorTree.CPP**: core library containing the basic functionality. It manages tree navigation and offers some basic default blocks as well as a plugin system.
- **Groot**: *BehaviorTree.CPP*-compliant graphical editor. It's possible to create, modify and monitor in real time trees made using the above-mentioned library.
- **ZeroMQ**: high-performance asynchronous messaging library. Not mandatory per se, but *Groot*'s real time monitoring relies on this.
- **behavior_tree_ros**: interface package between ROS and *BehaviorTree.CPP*. It offers seamless publication and subscription, a plugin containing some general nodes to manipulate messages and a general node to run trees.
- **Other plugins**: external plugins may offer new functionalities or they could serve as an interface for a given system. For example, a plugin for a particular robot could export publishers and subscribers for its custom messages. These plugins can be compiled and imported in *Groot*, thus allowing for an easy adoption. There is currently a repository in the group called **behavior_tree_plugins** where some plugins are already shipped.

Please note that, while *BehaviorTree.CPP* and *Groot* are external dependencies and have a public github repository maintained by the original author, Service Robotics Lab (SRL) has their own fork. Several new features still not implemented upstream have been added, and some upstream changes may not be completely supported yet by the rest of ROS-related packages. As such, the remaining of this document assumes the fork[1] is being used, and no distinction between custom features and those supported on the original repository is made.
Before going any further, install *ZeroMQ* using the following command:

Code Snippet 1: Installing dependencies

```
$ sudo apt install libzqm-dev
```

[1]SRL work is always added on the *upo* branch. *master* and *develop* branches are copies of their upstream counterparts and thus should be avoided.

BehaviorTree.CPP and *Groot* forks can be found in the group's repository under the same names. They are compiled as any other CMake project[2]:

Code Snippet 2: Compiling packages

```
$ mkdir build
$ cd build
$ cmake ..
$ make
```

BehaviorTree.CPP should be installed at the system level rather than just compiled. Consider installing using *sudo make install*. Finally, *behavior_tree_ros* and plugins in *behavior_tree_plugins* repos are normal ROS packages and follow the usual compilation procedure, but they depend on a few packages. Install them before trying to compile:

Code Snippet 3: Installing *behavior_tree_ros* dependencies

```
$ sudo apt install ros-kinetic-topic-tools ros-kinetic-ros-type-
    introspection
```

[2]Keep in mind that, while compilation using catkin is supported, its use is discouraged, due to some changes in the fork

3.2 Hello World!

Let's create our very own first tree. Programming tutorials usually starts off with a *Hello World!* example, and this document is no exception.

Start by creating a new catkin package. Your *CMakeLists.txt* should look like snippet 4.

Code Snippet 4: Importing *behavior_tree_ros* in cmake

```
1  cmake_minimum_required(VERSION 3.1 FATAL_ERROR)
2  project(behavior_tree_demo VERSION 1.0.0)
3
4  set(CMAKE_CXX_STANDARD 14)
5
6  if(NOT CMAKE_BUILD_TYPE)
7      set(CMAKE_BUILD_TYPE Release)
8  endif()
9
10 find_package(catkin REQUIRED COMPONENTS
11   roscpp
12   behavior_tree_ros
13   std_msgs
14 )
15
16 catkin_package(
17     CATKIN_DEPENDS
18         roscpp
19         behavior_tree_ros
20         std_msgs
21     LIBRARIES
22         ${PROJECT_NAME}
23 )
24
25 add_library(${PROJECT_NAME} src/plugin.cpp)
26 target_link_libraries(${PROJECT_NAME} ${catkin_LIBRARIES})
27 target_include_directories(${PROJECT_NAME} PRIVATE ${catkin_INCLUDE_DIRS
       })
28 target_compile_options(${PROJECT_NAME} PRIVATE -Wall -Werror -Wfatal-
       errors -Wl,--no-undefined)
29
30 install(TARGETS ${PROJECT_NAME}
31         LIBRARY DESTINATION ${CATKIN_PACKAGE_LIB_DESTINATION}
32 )
```

Fancy cmake details aside, the only thing that has to be done to use trees in ROS is to include the *behavior_tree_ros* package (see line 19). Now we have to include some code. While pretty much all the heavy lifting to publish, receive and (de)serialize messages is done *automagically* for you, the package needs to know at compile time what messages you intend to use. Line 25 compiles a library whose only source file is called *plugin.cpp*. Snippet 5 shows the contents of this file.

Code Snippet 5: Declaring a publisher in a plugin

```
1 #include <std_msgs/String.h>
2
3 #include <behaviortree_cpp/bt_factory.h>
4 #include <behavior_tree_ros/behavior_tree_ros.hpp>
```

```
5
6  BT_REGISTER_NODES(factory)
7  {
8      using namespace BT_ROS;
9
10     factory.registerNodeType<PublisherNode<std_msgs::String>>("
           DemoPublishString");
11 }
```

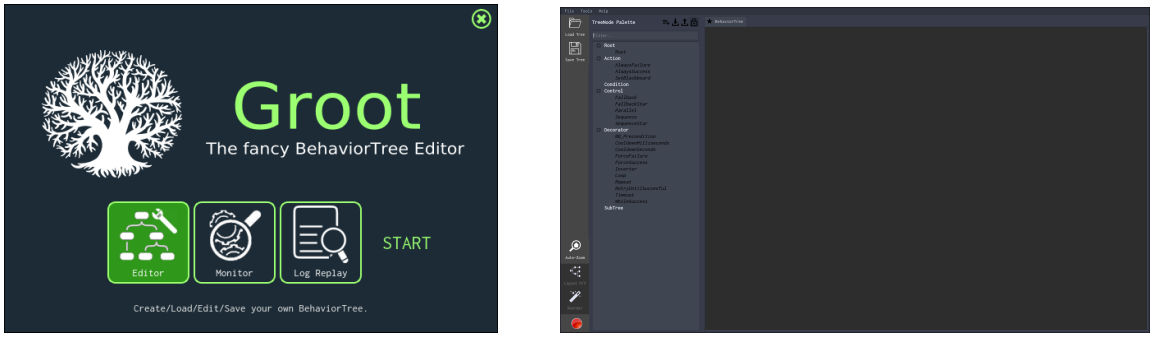
A standard string message publisher is instantiated in line 10. By using the templates exposed in the package *behavior_tree_ros* one can easily define a leaf node that will publish a given message every time is executed.

The code has to be compiled as a *BehaviorTree.CPP*-compliant plugin. A full explanation on how the plugin system works is out of the scope of this document (refer to the official online documentation instead [2]). Nevertheless, plugins always have this same exact boilerplate code, so the snippet can be used as a template.

We now have one of the building blocks of our behavior tree containing the functionality (in this case, a single *String* publisher). The next step is to define what blocks are interconnected and how should the tree be traversed. For this we are going to use *Groot*.

Launch it and select *Editor* in the splash screen (see Figure 2a). You should be presented with an empty workspace. On the left you can see the list of available blocks you may use to build your tree by dragging and dropping them into said workspace.

You can also define manually custom nodes by pressing the first button on top of the node list. A pop-up window should appear, where you can input the name, type and ports of the new node. This method is, however, tedious and error-prone, as the block definition you enter manually has to match those defined in the code of the block itself.



(a) Splash screen. (b) Empty workspace.

Figure 2: Groot splash screen and empty workspace

Rather than doing that, we are going to import the plugin you just compiled. This plugin not only contains the actual execution code, but also nodes' definition and all the info *Groot* needs. Click on the *Load Tree* button in the upper-left. From this menu you can import either behavior tree files in XML format (more on this later) or you can change the type in the bottom dropdown to plugins with a .so extension. The plugin should be in /path/to/workspace/devel/lib/.

After importing the plugin, the string publisher should have appeared in the list under the sames name used in the code (refer back to the snippet 5). The resulting tree is shown in Figure 3. You can find all the blocks you need in the list on the left, or you can type their names in the search bar on top of it.

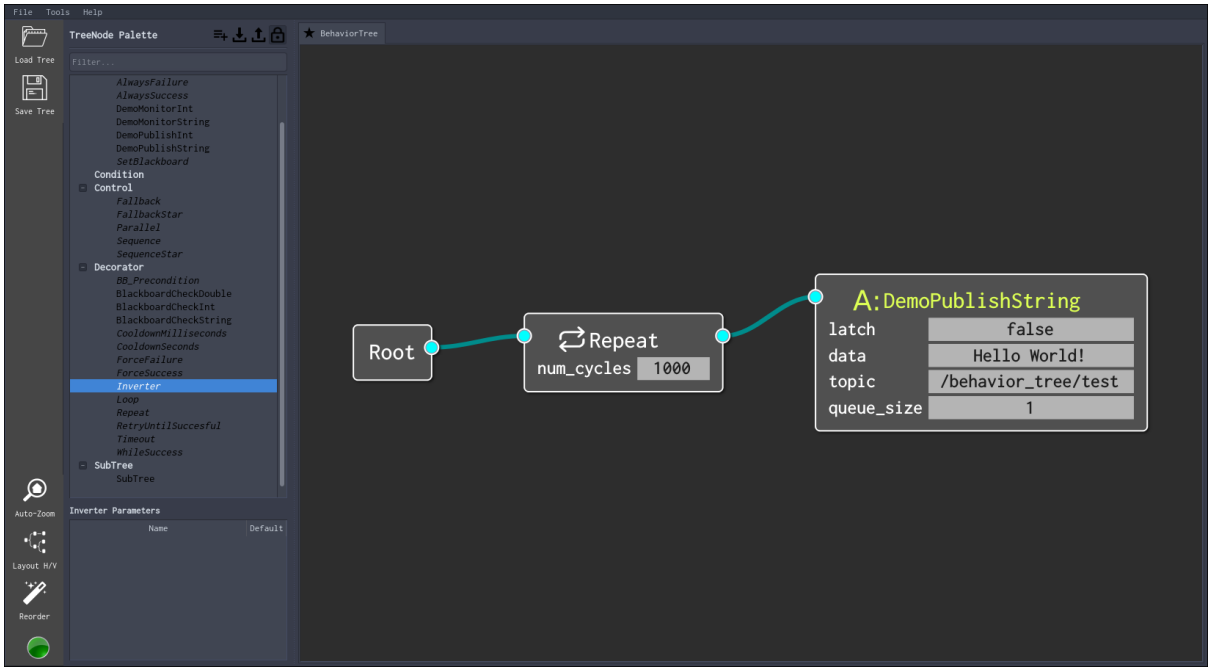


Figure 3: *Hello World!* tree

Let's take a closer look. The tree starts with a *root* node. Trees should always have a *root* as their first block. This connects with a *repeat* node, that has an input parameter to change the maximum number of tries. This block will keep ticking[3] its child as long as it returns SUCCESS. The *repeat* node itself will return SUCCESS if it has ticked successfully its child *num_cycles* times or FAILURE if not.

Finally, the last leaf node contains the actual user code. It will publish a message to the topic set in its port every time its ticked.

Click the save button on the left, and you will be prompted to save a file with an XML extension. The contents of the exported file should be similar to those of snippet 6. It's easy to spot the tree structure, defined in the section enclosed by the <BeahviorTree> tags. The <TreeNodesModel> tags declares the interfaces (input and output ports) of the non-default blocks present in *Groot* when the tree was exported, even if they were not actually used in the tree. The full XML schema and some documentation can be found in the official github repository.

Code Snippet 6: *Hello World!* tree XML file

```
1  <root main_tree_to_execute="BehaviorTree">
2      <!-- ////////// -->
3      <BehaviorTree ID="BehaviorTree">
4          <Repeat num_cycles="1000">
5              <Action ID="DemoPublishString" latch="false" data="Hello
                   World!" topic="/behavior_tree/test" queue_size="1"/>
6          </Repeat>
7      </BehaviorTree>
8      <!-- ////////// -->
9      <TreeNodesModel>
10         <Action ID="DemoPublishString">
```

[3]The term *tick* is used when a parent node calls the code contained in its child block. Internally, each block has a *tick()* function that defines what it actually does.

```
11              <input_port type="std::string" name="data">Auto-generated
                    field from std_msgs::String_&lt;std::allocator&lt;void>
                    ></input_port>
12              <input_port type="bool" name="latch" default="0">Latch
                    messages?</input_port>
13              <input_port type="unsigned int" name="queue_size" default
                    ="1">Internal publisher queue size</input_port>
14              <input_port type="std::string" name="topic">Topic to publish
                     to</input_port>
15          </Action>
16      </TreeNodesModel>
17      <!-- ////////// -->
18 </root>
```

Now we have the code defining what the nodes actually do (the plugin), and how these nodes are interconnected (the XML file). The next step is to execute it. The package *behavior_tree_ros* also includes a general node capable of executing trees. Before launching it, all the needed plugins should be placed in /path/to/behavior_tree_ros/resources/plugins/. At startup, the node will scan this folder for plugins and include all the nodes defined in them. Trying to run a tree that uses nodes not loaded will result in an error.

Once the plugin is placed in the directory[4], you can start the node using the following command:

Code Snippet 7: Launching *behavior_tree_ros* node

```
$ roslaunch behavior_tree_ros behavior_tree_node.launch
```

You should see a message stating that it managed to load the plugin. Trees can be run by importing the XML file into the node. Launch the tree with the command below:

Code Snippet 8: Loading a tree

```
$ rosservice call /behavior_tree/load_tree "tree_file: '/path/to/tree/
    xml/file'"
```

If everything went fine, you should get some output in the terminal, with the current block in the tree being executed and its result. You can check using rostopic that messages containing the string *Hello World!* are being published in the topic you input in the publisher node's port.

While this tree is far from fancy, the example illustrates the basic work-flow and how to proceed when introducing trees in a new ROS system. To sum it up:

- Define your leaf nodes in a plugin. They may be publishers and subscribers using custom messages or they can perform some other operations/checks.
- Import all the plugins containing the blocks you need in Groot.
- Build your tree and export it as an XML file.
- Put all the used plugins in the default ROS node folder so they are loaded at start up.
- Load the XML file using the ROS service.

And that's all. No matter how big your tree may be, these steps are always the same.

[4]You can use symlinks too, so you don't have to copy the plugins over every time you recompile them.

3.3 Adding Logic to your Trees

Let's spice things up a bit. To build more complex behaviors we need input data. This data usually take the form of ROS messages received from topics we would like to monitor. We also need to make decisions based on this information, usually by checking values and exploring the data inside the message.

One approach to this is to build custom leaf nodes that work on messages of a given message type. This does not escalate so well, as it would mean you would have to write custom nodes every time you want to manipulate some message. To avoid this, *behavior_tree_ros* subscriber leaf nodes can serialize the messages they receive into a more general structure, allowing for more general-purpose smaller blocks capable of inspecting and performing operations on these messages.

Let's put this to test by building a simple tree that, given a custom user message containing two numbers, it tells us which one is bigger. For the sake of the example, we are going to assume that whoever defined the message had a bad day and did it as shown in the snippet 9. But worry not, for introspection is here to save the day.

Code Snippet 9: *TwoNumbers* message definition

```
Header header
std_msgs/Float32[2] numbers
```

Add the message to your package and cmake file. A new leaf node to subscribe to this new message should be added in the plugin as well. Modify the *plugin.cpp* file as shown in snippet 10.

Code Snippet 10: Declaring a subscriber in a plugin

```
1  #include <std_msgs/String.h>
2  #include <behavior_tree_demo/TwoNumbers.h>
3
4  #include <behaviortree_cpp/bt_factory.h>
5  #include <behavior_tree_ros/behavior_tree_ros.hpp>
6
7  BT_REGISTER_NODES(factory)
8  {
9      using namespace BT_ROS;
10
11     factory.registerNodeType<PublisherNode<std_msgs::String>>("
           DemoPublishString");
12
13     factory.registerNodeType<SubscriberNode<behavior_tree_demo::
           TwoNumbers>>("DemoMonitorTwoNumbers");
14 }
```

Recompile it and import it in Groot. The new tree containing the logic to compare the numbers is shown in figure 4. Things have gotten a bit more complicated now, so let's go step by step.

You may notice that there are several new blocks such as *GetMessageField* or *Loop* that may not appear in you block list. If that's the case, you have to import the plugins containining the definitions. For this example, you have to import the plugin defined in the *behavior_tree_ros* package. It exports some nodes used to introspect ROS messages. In the *behavior_tree_plugins* repo you will find several packages with plugins as well. You

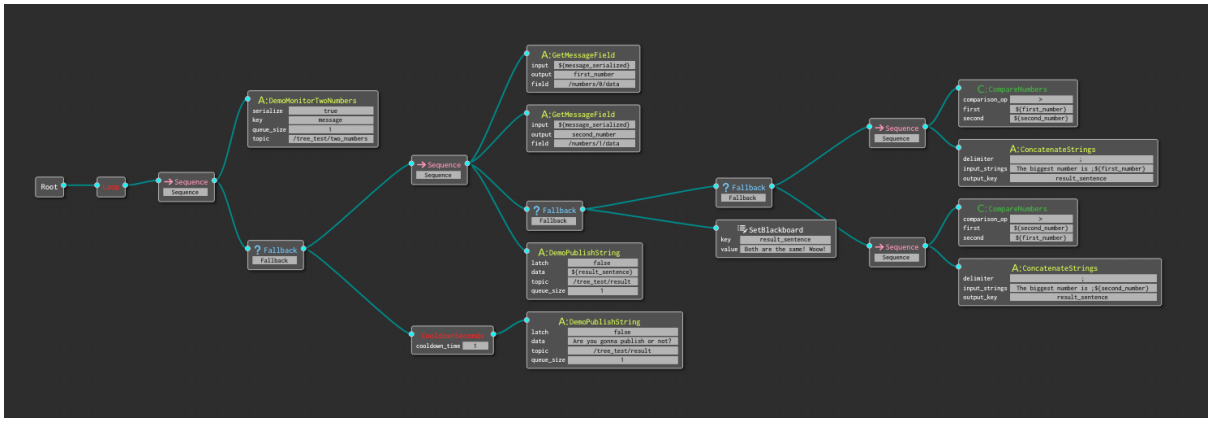


Figure 4: Tree comparing two numbers

need the one in the package *behavior_tree_extensions* to be able to use blocks such as *CompareNumbers*.

Let's take a look now at the actual structure of the tree. The loop node at the beginning just repeat the execution of the tree indefinitely, no matter what its child returns. After that, a subscriber of our custom message is instantiated. New messages will be stored under whatever variable name you put in the *key* port. You will also get a serialized version of the message under the name *${key}_serialized* if you set the *serialize* port to true (it's enabled by default). You can always opt-out of the serialization and write your own nodes that receive and process messages of a given type in their ports. However, serialized messages allow you to introspect and make decisions based on values without having to write any additional code.

Also note that subscribers nodes do not have to be executed on every tick for messages to be received. The node will subscribe to the topic during its initialization and it will work on parallel. It returns always SUCCESS if it ever gets ticked, so it doesn't hurt to just leave them inside a loop.

Sequences and fallbacks are basic flow control nodes. Think of sequences as "do these tasks in order unless one of them fails". Fallbacks, on the other hand, will execute the next child only if the previous one fails, so you can think of them as an if-else.

Following the upper branch we see there's a node called *GetMessageField*. This is a basic introspection block, that allows to access an element of a serialized ROS message and save it to a variable. There are two important things to note here. First, the syntax used to access a variable. You can always refer to a variable in an input port using the syntax ${...}[5].

The other important thing to note is the way serialized fields are accessed. In the first block, we access the first element of the array with the pointer */numbers/0/data*. You may have noticed that *numbers* and *data* are the same field names used in the definition of the custom message we created before. This is no coincidence. To access any element of any serialized ROS message you only have to take a look at the message definition. Indexes can be used as well to access a particular element of an array. For more information on how this works refer to annexe A.

We now have our two parsed numbers in two variables we can use to make a decision. The branches after the fallback do just that. It tries to compare both of them and set an appropriate sentence. If it fails, it means the values were the same, and it defaults to

[5]In fact, you can even add more indirection levels by nesting the accessors like this ${${...}}.

a predefined reply. After that, the sentence with the result is published using the same node that was used in the *Hello World!* example.

Keep in mind that type safety is ensured when accessing and extracting elements from a serialized message, and in general in all the other nodes. This means you should be careful with the types nodes expect in their input ports, and what are the types of the message fields you are accessing when passing information from node to node. Newer versions of *BehaviorTree.CPP* will support type safety for port connections, but for the time being it's responsibility of the user.

The second branch of the outer fallback only gets executed if the upper branch fails, that is, if the accessors nodes fail. This probably means no message has being published yet, so we let know the user how upset we are about that. The only new thing here is the *cooldown* before the publisher. Cooldown blocks limit the maximum rate its child node is ticked. If it gets ticked at a rate faster than the limit, it will report back the value returned by its child the last time it was ticked. Only after enough time has passed it will execute it again.

Timing is something you always have to be careful about when designing a tree. The tree is executed (that is, traversed and its nodes ticked) at a given fixed rate that it's probably higher than the speed you want to send commands. Cooldown and sleeps are the two common mechanisms to control timing in a tree.

You can export this tree, launch the node and load the XML file. Before trying to publish any message, check the output of the */tree_test/result* topic. You should get passive-aggressive "*Are you gonna publish or not?*" messages at a rate of one second. Let's send two numbers for the tree to compare.

Note that you may get an "*Unknown type*" error if you try using rostopic pub on the */tree_test/two_numbers* topic and get tab auto-completion. This is because there are no publishers connected to that topic yet. This is a limitation of ROS itself, not of the tree. As a workaround, you can type manually the message tyep (*behavior_tree_test/TwoNumbers*) and try to send and empty string. You will get an error, but the next time you try to tab you should get auto-completion. Try sending different numbers and check that you get the expected output.

And that's it. We have gone a bit further in this section, taking some simple decisions based on values of a message published in a topic, all of this with just one additional line of code.

3.4 Logging and Monitoring

At this point you may be wondering how to monitor the execution of a tree. *BehaviorTree.CPP* offers several type of loggers to accomplish this, but we are going to focus on the real-time monitoring features. You can check the launch file in the *behavior_tree_ros* package and see what options are available (refer to the online docs for a more in-depth explanation).

There should be two loggers in the launch file enabled by default: the command line (cout) and the zmq logger. Command line monitoring can be quite overwhelming when your tree grows in size. The zmq one is based in the *ZeroMQ* library. It will send messages containing the current status of the tree. These messages are processed by *Groot*, allowing to monitor a tree in real time.

Let's check how it works. Run again the tree we used to compare two numbers back in section 3.3. After that, fire up a new instance of *Groot* and select *Monitor* in the splash

screen. Click on connect and the tree structure should pop up, similar to figure 5.

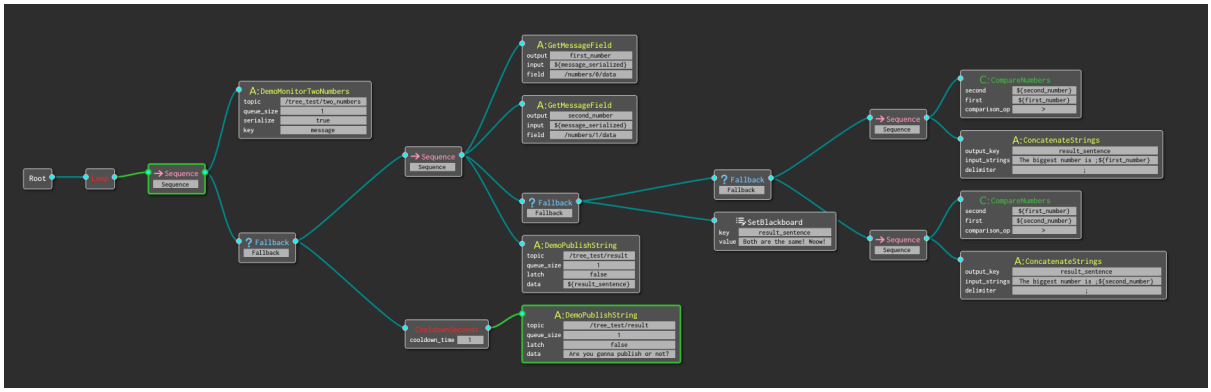


Figure 5: Real time tree monitoring with *Groot*

The return status of each block will be represented by the color of the connection to its parent. Green means SUCCESS, red means FAILURE and a orange connection indicates that the block is returning RUNNING. Note that this feature is still a bit rough in the edges, so it may be difficult to know what's exactly going on sometimes.

3.5 Adding Memory to your Trees

The examples we have seen until now are said to be memoryless. This means that, every time a control node such as a sequence or a fallback was ticked, it ticked its children again from the beginning. This was not a problem in the previous trees, as all the nodes returned either SUCCESS or FAILURE immediately. There are, however, times when a node may return RUNNING as a status, either because its performing a task that takes a while to finish or because it's trying to control the timing/flow of the tree itself.

Consider the tree in figure 6. In each iteration, a new task is commanded and the processing is halted until it finishes. Let's assume also that the *WaitForTaskToFinish* node returns RUNNING while the task is not finished. As it is right now, the tree is ill-formed and it will not work as expected. As soon as the wait node returns RUNNING, the sequence node will report back that to its parent. The loop will just tick the sequence again after that. However, as stated before, this sequence is memoryless, so in this new tick, it will start again the execution of its children from the beginning. This means a new task will be executed before the previous one is finished.

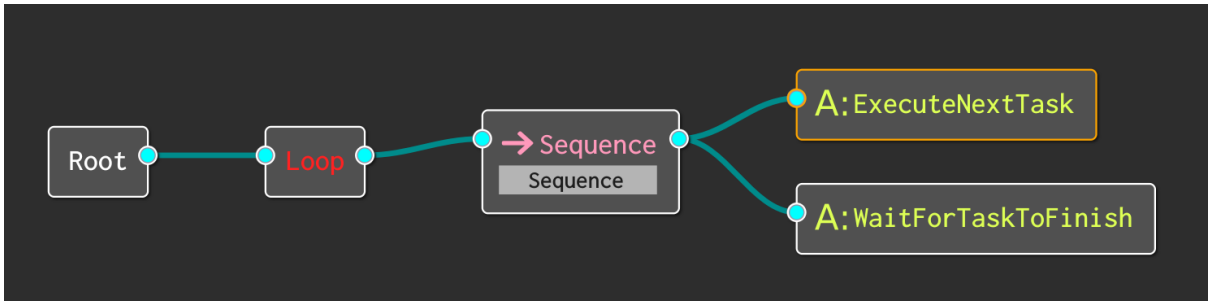


Figure 6: Ill-formed tree with no memory

This is obviously not what we want. Figure 7 shows how to fix this. The vanilla sequence has been changed by its *star* counterpart. All major control blocks have star

versions with memory. Now, when the loop ticks again, the sequence will remember that one of its children returned RUNNING before, and it will tick it again to see if it has finished. This process will be repeated until it returns either SUCCESS or FAILURE.

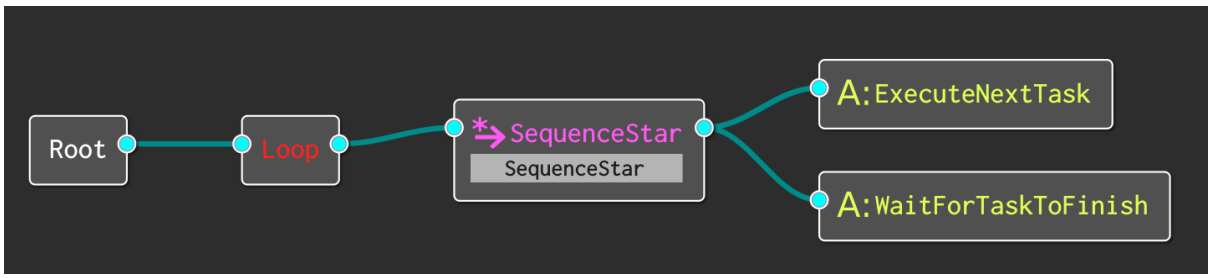


Figure 7: Correct tree using memory

After seeing this you may be tempted to add a control with memory every time you have a node that can return RUNNING somewhere under it. However, this can have unexpected results. Let's supposed now that we have to check the temperature levels of the system. If it ever gets above a threshold, missions should be aborted and the system should be turned off to ensure its integrity Figure 8 depicts a possible implementation of this.

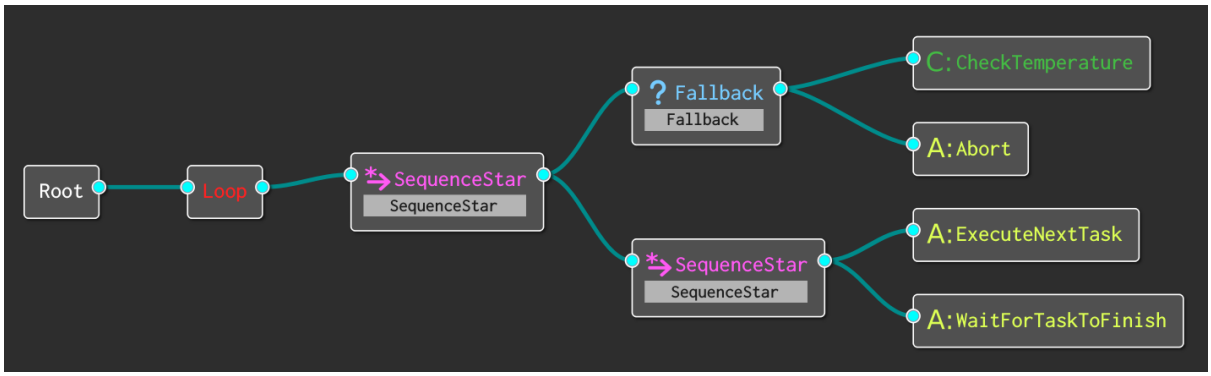


Figure 8: Tree lacking responsiveness due to bad memory placement

CheckTemperature will return SUCCESS only if the system temperature is under the limits. If something goes wrong, it will return FAILURE, the fallback will switch to its second branch and the *Abort* node will be executed.

Can you spot the issue here? The problem stems from the fact that both sequences have memory. The branch checking for the temperature will never be called while the system is waiting for a task to finish, as the outer sequence remembers one of its children returned RUNNING. Only after the task is finished and the external sequence is reseted the temperatures will be checked. Depending on how long the tasks are this could pose a problem or not.

The fix is rather simple, though. Simply switching the first sequence to its memoryless version does the trick. Figure 9 shows the final result. Now, in every iteration the temperature will be checked first and after that the tasks branch will be executed, continuing from where it was left in the last iteration (either waiting for the current one to finish or commanding the next one).

As seen in the above examples, special care should be taken when adding memory to a tree, as a bad placement of the *star* control nodes can reduce the responsiveness of the

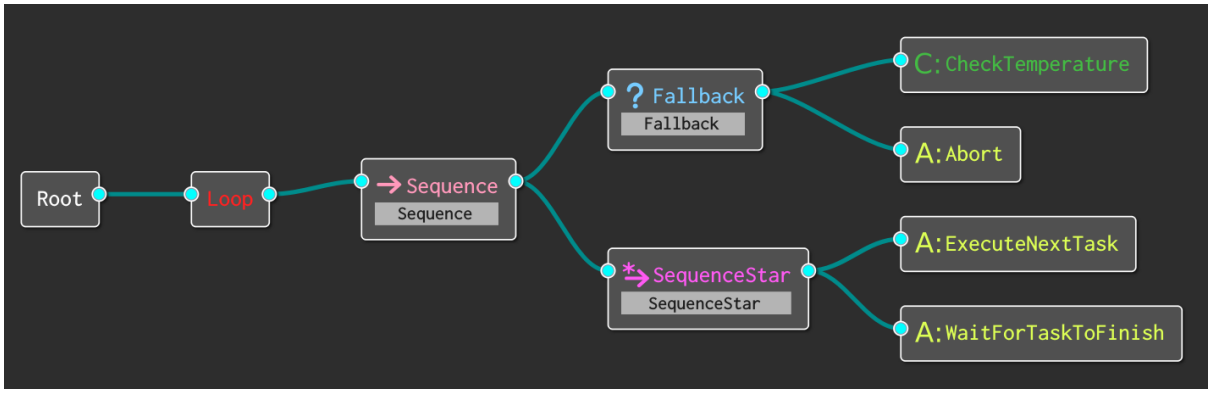


Figure 9: Proper multi-level memory management example

behavior against important events dramatically. Placing blocks with memory correctly is the key to avoid this.

3.6 Using Subtrees

One of the major benefits of behavior trees is their high re-usability. Thanks to a simple interface (a node returns either SUCCESS, FAILURE or RUNNING), trees can be reused in other trees easily. This becomes even more important when the approach taken in the design is to use smaller general blocks along with introspection to execute basic operations on the input data, as shown in section 3.3. With this approach, big blocks containing chunks of code tailored to a specific task are substituted by subtrees made up of several simpler blocks.

Let's go back to the example of the two numbers presented in section 3.3. We would like to put all the logic to compare the two numbers inside its own subtree. This subtree would take as an input two numbers and will output a string containing the result of the comparison.

To create the new subtree right-click on the second *fallback* node, just after the two *GetMessageField* nodes and select the "*Create Subtree Here*" option. And *voilà*, you now have a valid subtree[6]. All the blocks containing the comparison should have been substituted by a subtree block, as in figure 10. You can expand/collapse it in-situ, or you can inspect its contents by clicking on the tabs on top of the workspace.

As stated before, the tree expects you to feed it two numbers in two different variables, and it gives you the result in another one. As it stands right now, there's no way to know what input and output ports a subtree has just by looking at the external block.

There are no distinctions made either between internal and external variables, so be careful when naming your variables. If you are not careful, you may end up overwriting internal variables of a subtree. Future versions of *BehaviorTree.CPP* will account for this, and a port system similar to that of SMACH will be included to avoid name-clashing.

Remember also that all subtrees are available in the block list on the left, so you can instantiate them as many times as you want.

Let's take a look at the generated XML file to see the differences. Snippet 11 shows the contents of the file, where the *TreesNodesModel* section has been truncated for clarity. It's easy to see that the only addition to the file is a new *BehaviorTree* section with the

[6]In fact, this is currently the only way to create subtrees in *Groot*. There's no method to create a new subtree from scratch, and trying to expand and empty subtree block will crash the application.

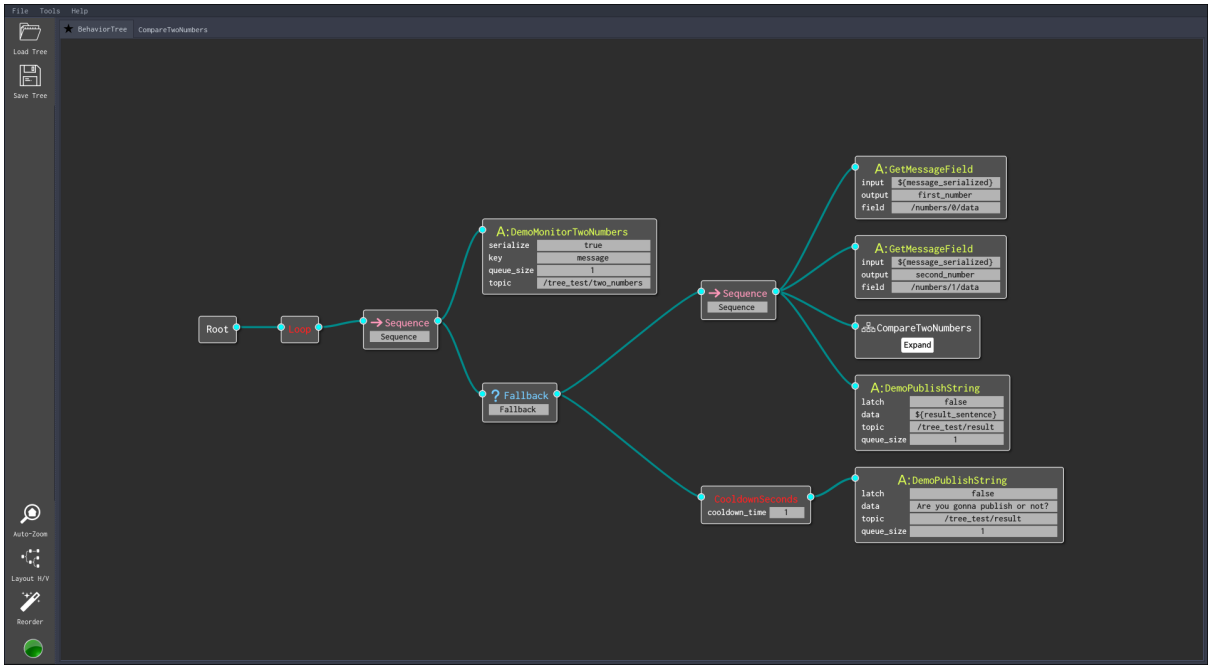


Figure 10: Tree containing a subtree

definition of the new tree. XML files should always have a root tree acting as the entry point. Line 1 declares the main tree to execute. Other trees defined in the file will be treated as subtrees.

Code Snippet 11: Defining a subtree in a XML tree file

```
1  <root main_tree_to_execute="BehaviorTree">
2      <!-- /////////// -->
3      <BehaviorTree ID="BehaviorTree">
4          <Decorator ID="Loop">
5              <Sequence>
6                  <Action ID="DemoMonitorTwoNumbers" serialize="true" key
                       ="message" topic="/tree_test/two_numbers" queue_size
                       ="1"/>
7                  <Fallback>
8                      <Sequence>
9                          <Action ID="GetMessageField" serialize="${
                               message_serialized}" output="first_number"
                               field="/numbers/0/data"/>
10                         <Action ID="GetMessageField" serialize="${
                               message_serialized}" output="second_number"
                               field="/numbers/1/data"/>
11                         <SubTree ID="CompareTwoNumbers"/>
12                         <Action ID="DemoPublishString" latch="false"
                               data="${result_sentence}" topic="/tree_test/
                               result" queue_size="1"/>
13                     </Sequence>
14                     <Decorator ID="CooldownSeconds" cooldown_time="1">
15                         <Action ID="DemoPublishString" latch="false"
                               data="Are you gonna publish or not?" topic="/
                               tree_test/result" queue_size="1"/>
16                     </Decorator>
17                 </Fallback>
18             </Sequence>
```

```
19          </Decorator>
20      </BehaviorTree>
21      <!-- /////////// -->
22      <BehaviorTree ID="CompareTwoNumbers">
23          <Fallback>
24              <Fallback>
25                  <Sequence>
26                      <Condition ID="CompareNumbers" first="${first_number
                            }" comparison_op="&gt;" second="${second_number
                            }"/>
27                      <Action ID="ConcatenateStrings" delimiter=";"
                            input_strings="The biggest number is ;${
                            first_number}" output_key="result_sentence"/>
28                  <Sequence>
29                  <Sequence>
30                      <Condition ID="CompareNumbers" first="${
                            second_number}" comparison_op="&gt;" second="${
                            first_number}"/>
31                      <Action ID="ConcatenateStrings" delimiter=";"
                            input_strings="The biggest number is ;${
                            seconds_number}" output_key="result_sentence"/>
32                  <Sequence>
33              </Fallback>
34              <SetBlackboard value="Both are the same! Woow!" key="
                    result_sentence"/>
35          </Fallback>
36      </BehaviorTree>
37      <!-- /////////// -->
38      <TreeNodesModel>
39          <Decorator ID="Loop"/>
40          <Decorator ID="CooldownSeconds">
41              <input_port default="0" type="unsigned int" name="cooldown">
                    Cooldown time</input_port>
42          </Decorator>
43           <Condition ID="CompareNumbers">
44              <input_port type="std::string" name="comparison_op">
                    Comparison operator. Valid operators are &lt;, >, &lt;=,
                    >=, == and !=</input_port>
45              <input_port type="double" name="first">First operand</
                    input_port>
46              <input_port type="double" name="second">Second operand</
                    input_port>
47          </Condition>
48          <Action ID="ConcatenateStrings">
49              <input_port type="std::string" name="delimiter"></input_port
                    >
50              <output_port type="std::string" name="output_key"></
                    output_port>
51              <input_port type="std::string" name="input_strings"></
                    input_port>
52          </Action>
53          <Action ID="GetMessageField">
54              <input_port type="std::string" name="field">Field to fetch</
                    input_port>
55              <input_port type="nlohmann::basic_json&lt;std::map, std::
                    vector, std::__cxx11::basic_string&lt;char, std::
                    char_traits&lt;char>, std::allocator&lt;char> >, bool,
                    long, unsigned long, double, std::allocator, nlohmann::
```

```
                adl_serializer>" name="input">Serialized ROS message</
                input_port>
56          <output_port type="BT::Any" name="output">Output variable</
                output_port>
57      </Action>
58      <Action ID="DemoPublishString">
59          <input_port type="std::string" name="data">Auto-generated
                field from std_msgs::String_&lt;std::allocator&lt;void>
                ></input_port>
60          <input_port type="bool" name="latch" default="0">Latch
                messages?</input_port>
61          <input_port type="unsigned int" name="queue_size" default
                ="1">Internal publisher queue size</input_port>
62          <input_port type="std::string" name="topic">Topic to publish
                 to</input_port>
63      </Action>
64      <Action ID="DemoMonitorTwoNumbers">
65          <input_port type="std::string" name="key"></input_port>
66          <input_port type="bool" name="serialized" default="0"></
                input_port>
67          <input_port type="unsigned int" name="queue_size" default
                ="1"></input_port>
68          <input_port type="std::string" name="topic"></input_port>
69      </Action>
70      <SubTree ID="CompareTwoNumbers"/>
71  </TreeNodesModel>
72  <!-- /////////// -->
73 </root>
```

3.7 Writing your own nodes

While manipulating ROS messages is fine and dandy, there are times you would like to write your own nodes (be it a leaf, control or decorator) to accomplish a given task or to add a new more general operation that was missing from the library. The online documentation is the main resource for this [2].

Just some heads-up. Be careful when changing and reporting back result statuses. Your custom nodes shall never return IDLE. Control nodes will freak out and throw an exception, halting the whole tree. It's responsibility of the control nodes to set its children status back to IDLE, so you don't have to worry about that.

Also, avoid doing any sleep or performing any function that may take a while to finish. If you need to perform a long task, your node should return RUNNING immediately and run computations in parallel. Consider using techniques described in the online documentation such as coroutines or async actions.

4 Nodes Reference

Decoupling nodes code from the layout of the tree allow users to build trees using external graphical tools such as *Groot*. However, it's sometimes difficult to understand exactly what a node does and when it should or shouldn't be used just by looking at its parameters in the GUI. You end up checking the code to fill in the gaps, which kind of defeats the point.

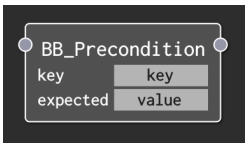
This section aims to present an exhaustive list of all the nodes currently available at SRL. Nodes are classified by its nature or the system they are targeted at. Some nodes are available by default with *BehaviorTree.CPP*, while others are more ROS-specific. There are also adaptations of nodes implemented by third-party well-known behavior trees frameworks, like the *Unreal Engine Behavior Tree Editor* [3].

A summary describing what they do is provided, as well as input and output ports and expected variable types.

4.1 Default nodes

These nodes are pre-built with *Groot* and *BehaviorTree.CPP*. They are always available, even if no plugin has been imported.

4.1.1 Decorator nodes

Block	Description	Ports
	Returns SUCCESS if a given key exists in the blackboard and has an expected value.	**key** Entry to check. **expected** Expected value (* for any).
	Returns FAILURE always, independently of its child's result.	No ports.
	Returns SUCCESS always, independently of its child's result.	No ports.
	Returns FAILURE if its child returns SUCCESS and SUCCESS otherwise (RUNNING status is not changed).	No ports.
	Ticks its child a set number of times as long as it returns SUCCESS. Returns FAILURE and stops the cycle otherwise.	**num_cycles** Maximum number of repetitions.

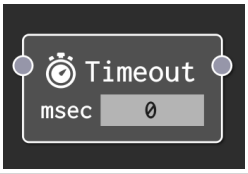
	Keeps ticking its child until it returns SUCCESS.	**num_attemps** Maximum number of tries.
	Returns FAILURE and halts its child if it keeps returning RUNNING after a set time.	**msec** Timeout in milliseconds.

Table 2: Default decorators nodes.

4.1.2 Composite nodes

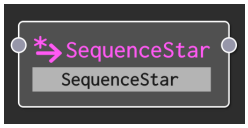
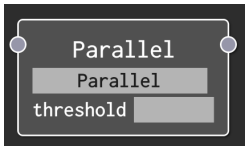
Block	Description	Ports
	Ticks its children in order. Returns SUCCESS if all of them return SUCCESS, RUNNING if any of them return it and it returns FAILURE and halts if any of them fails.	No ports.
	Sequence block with memory.	No ports.
	Returns SUCCESS and stops when one of its children returns SUCCESS, ticks the next one otherwise.	No ports.
	Returns SUCCESS always independently of its child's result.	No ports.
	Ticks all its children at the same time. Returns FAILURE and halts the process if a set number of children returns it.	**threshold** Maximum number of children that can fail.

Table 3: Default composite nodes.

4.1.3 Leaf nodes

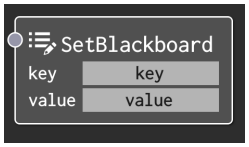
Block	Description	Ports
	Returns FAILURE always.	No ports.
	Returns SUCCESS always.	No ports.
	Inserts a new entry in the blackboard.	**key** Name of the new entry. **value** Value of the new entry.

Table 4: Default leaf nodes.

4.2 Introspection nodes

These nodes can be found in the *behavior_tree_ros* package plugin. They offer message-agnostic operations and operates on serialized ROS messages.

4.2.1 Decorator nodes

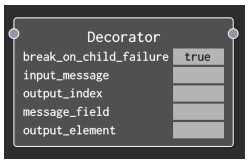
Block	Description	Ports
	Node implementing a *forEach* loop over a sequence. For each entry in the input sequence it will tick its child.	**break_on_child_failure** If true, it will return FAILURE and halt the loop if its child returns FAILURE. **input_message** Serialized ROS message input. **message_field** JSON pointer to the sequence in the ROS message to iterate through. **output_index** Optional variable name used to save the current sequence entry index. **output_element** Optional variable name used to save the current sequence entry element.

Table 5: Introspection decorator nodes.

4.2.2 Leaf nodes

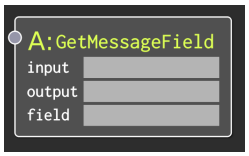
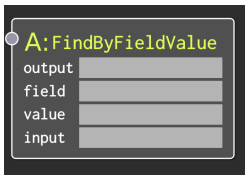
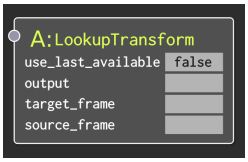
Block	Description	Ports
	Extracts a field from a serialized ROS message. Type safety ensured.	**input** Serialized ROS message input. **field** JSON pointer to the field to extract. **output** Output variable name.
	Extracts a random entry from a sequence.	**input** Serialized ROS message input. **field** JSON pointer to the sequence. **output** Output variable name.
	Finds an entry in a sequence with a given value.	**input** Serialized ROS message input. **field** JSON pointer to the sequence. **output** Output variable name. **value** Value to search.

Table 6: Introspection leaf nodes.

4.3 TF nodes

Some TF-related nodes are available in the *behavior_tree_extensions* package plugin.

4.3.1 Leaf nodes

Block	Description	Ports
	Queries the TF system for a transform between two frames. The resulting TF message will not be serialized.	**use_last_available** If true, use last transform available. Search using the current system time otherwise. **output** Variable used to save the result. **target_frame** Target TF frame. **source_frame** Source TF frame.

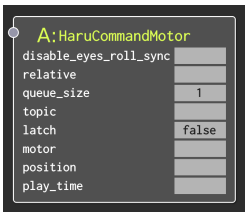
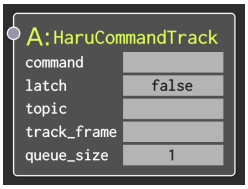
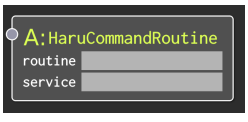
	Computes the 3D distance of a TF transform.	**input** Input TF transform message (not serialized). **output** Variable used to save the result.
	Computes the 2D distance (distance over the X-Y plane) of a TF transform.	**input** Input TF transform message (not serialized). **output** Variable used to save the result.

Table 7: TF leaf nodes.

4.4 Haru nodes

Nodes specific to Haru. Available in the *behavior_tree_haru* package plugin. Ports related to the publisher (topic, latch and queue_size) or to service clients (service name) are not included in the table.

4.4.1 Leaf nodes

Block	Description	Ports
	Publishes a *MotorCommand* message.	**disable_eyes_roll_sync** Disable LCD eyes video and roll sync. **relative** Set command relative to current position. **motor** Motor ID [0-4]. **position** Target position in radians. **play_time** Movement duration [0-250].
	Publishes a *TrackCommand* message.	**command** Command ID: 0 start tracking and 1 to stop. **track_frame** TF frame to track.
	Commands a routine using the *id-mind_tabletop/execute_routine* service.	**routine** Routine ID.

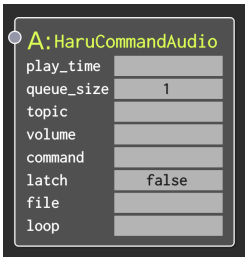
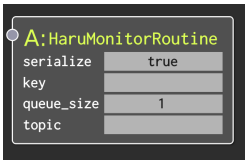
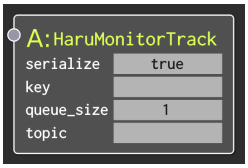
	Publishes an *AudioCommand* message.	**command** Command ID: 0 start playing the file and 1 to stop. **file** Audio wave file to play. **volume** Volume level [0.0-1.0]. **loop** Loop file. **play_time** Duration in milliseconds the file should be played (0 means the whole file).
	Subscribes to a *RoutineStatus* message type topic.	Subscriber's default ports.
	Subscribes to a *TrackStatus* message type topic.	Subscriber's default ports.

Table 8: Haru leaf nodes.

4.5 Strawberry nodes

Strawberry-related nodes. Available in the *behavior_tree_strawberry* package plugin. Ports related to the publisher (topic, latch and queue_size) or to service clients (service name) are not included in the table.

4.5.1 Leaf nodes

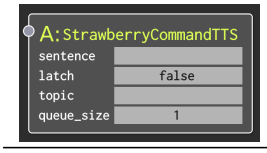
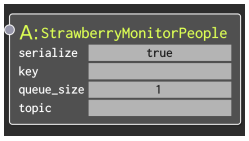
Block	Description	Ports
	Publishes a *TTSCommand* message.	**sentence** String containing the sentence to send to the TTS system.
	Subscribes to a *People* message type topic.	Subscriber's default ports.

Table 9: Strawberry leaf nodes.

4.6 Misc nodes

This section contains nodes that cannot really be put into any other category, such as waits, loops or comparisons. They are basic basic building blocks that are used in pretty much

any behavior tree. Keep in mind that they do not rely in ROS messages introspection.

4.6.1 Decorator nodes

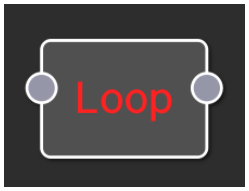
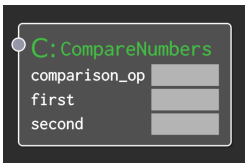
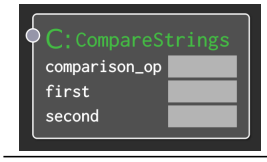
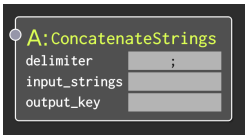
Block	Description	Ports
	Sets a maximum tick rate for its child. After being ticked, it will keep returning its child last return value until the cooldown time has passed.	**cooldown_time** Cooldown time.
	Keeps ticking its child independently of its return value.	No ports.
	Keeps ticking its child as long as it returns SUCCESS.	No ports.

Table 10: Misc decorator nodes.

4.6.2 Leaf nodes

Block	Description	Ports
	Compare two numbers (integer or float) using the comparison operator set by the user. Returns SUCCESS if the comparison is true or FAILURE otherwise.	**comparison_op** Comparison operator. Valid values are: ==, !=, <, <=, > and >=. **first** Left operand. **second** Right operand.
	Comparison block (strings version).	Same as *CompareNumbers* node.
	Concatenate the strings in the input sequence.	**delimiter** Input sequence entries delimiter (defaults to ';'). **input_strings** Sequence containing the strings to concatenate. **output_key** Output variable.

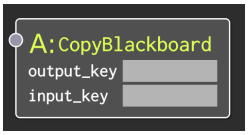
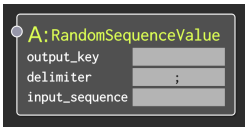
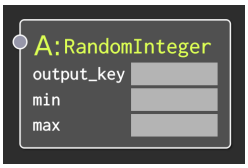
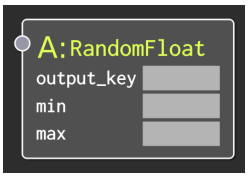
	Copies the contents of one blackboard's entry to another.	**input_key** Input variable. **output_key** Output variable.
	Gets a random entry from an input sequence.	**delimiter** Input sequence entries delimiter (defaults to ';'). **input_sequncee** Input sequence. **output_key** Output variable.
	Returns a random integer in the range [min,max].	**min** Range minimum value. **max** Range maxximum value. **output_key** Output variable.
	Returns a random integer in the range [min,max].	Same as *RandomInteger* node.
	Returns RUNNING for a set amount of time. Returns SUCCESS when it's done.	**duration** Wait time in milliseconds.

Table 11: Misc leaf nodes.

5 Behavior Trees Cookbook

Even though trees are rather simple in concept, thinking in terms of them not so much. There might be times you hit a dead wall trying to express some logic, unable to find the right connection between decorator, control and leaf nodes that gives you the logic flow you need.

This section aims to mitigate that, providing some common general structures and tricks you can use in your own trees. This is a constant work-in-progress. The more we work with trees, the more it will grow with good practices and know-how.

5.1 While Loops

While loops are one of the basics way to control the flow of a program. There are times you may have the need to express this same concept of "*while X do Y*" in your trees. Let's use a real example to illustrate how to achieve this.

Haru, the social robot in the lab offers several ways to interact with it, one of them being via physical gestures. When the person currently engaged (that is, being tracked) does a gesture, it should do some animation as a response. However, for security reasons, tracking should be halted while the animation is playing. Basically we have a "*while the tracked person is present do ...*" structure.

There are mainly two ways to express this. Figure 11 shows a first alternative. Here we are exploiting the fact that memory is always local to a block (this was explained in section 3.5). The condition is put in the first branch and the loop body in the second. The first sequence has no memory, so the *IsTrackedPersonPresent* condition will always be checked first. After that, the second branch, containing the gesture detection and animation response is executed from where it was left in the last iteration.

Retry and *Timeout* blocks are added at the beginning to get out of the loop if the person does nothing during a preset amount of time.

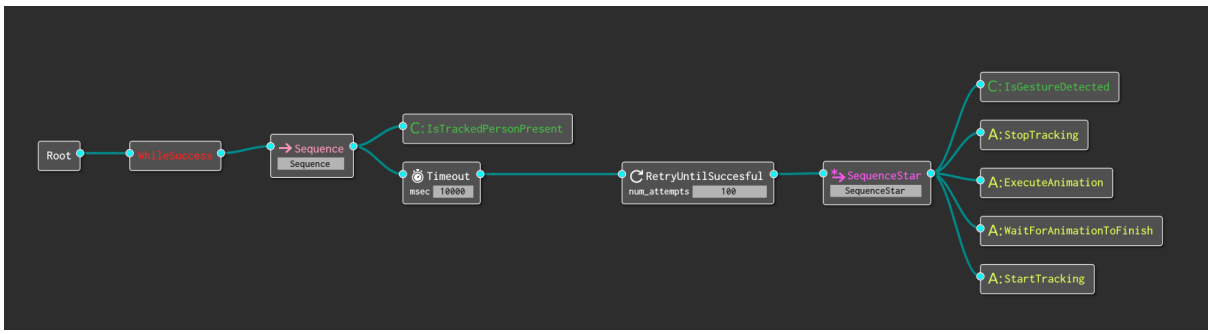


Figure 11: Implementing a while loop in a tree

The second method may be a bit more easy to understand visually. Figure 12 shows it. Here, the condition and the loop body are split in the two branches of the initial *Parallel* block. This block will tick alternatively its children. The parameter *threshold* is the number of branches that have to return SUCCESS for the whole parallel block to report SUCCESS back to its parent.

In other words, with a threshold of 2 and 2 connected branches, we have that 0 branches can return FAILURE before all processing is halted. If any of the two branches fails we get out of the loop. The two *WhileSuccess* in each branch ensure that we keep looping while the conditions are true.

Both versions are equivalent, as there's no overhead to the *Parallel* node.

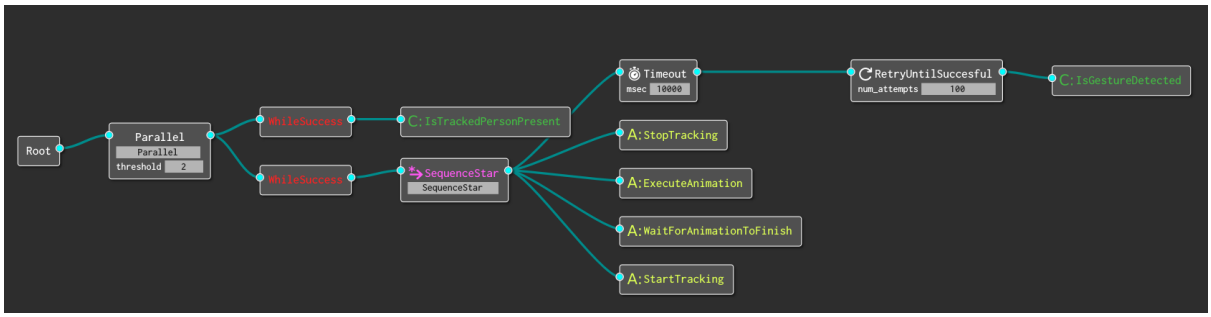


Figure 12: Alternative way to implement a while loop in a tree

6 Conclusions

Behavior trees present the flexibility, modularity and re-usability necessary to build a general behavior framework. This would speed up development times and unify the way behavior and decision-making in general is built in SRL. By generalizing ROS structures, it's possible to develop message-agnostic nodes containing the most basic operations, thus avoiding ad-hoc applications tailored to the specific needs of a given project.

References

[1] M. Colledanchise and P. Ögren, "How Behavior Trees Modularize Hybrid Control Systems and Generalize Sequential Behavior Compositions, the Subsumption Architecture, and Decision Trees," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 372–389, April 2017.

[2] D. Faconti, "Behaviortree.cpp documentation," 2019. [Online]. Available: https://behaviortree.github.io/BehaviorTree.CPP/

[3] 2019. [Online]. Available: https://docs.unrealengine.com/en-us/Engine/AI/BehaviorTrees/NodeReference

A ROS Messages Introspection

As it was discussed in section 3.3, *behavior_tree_ros* package offers publishers and subscribers leaf nodes that can manipulate ROS messages without the need to write any additional line of code. This annexe contains a technical description on how this works, as well limitations and workarounds.

ROS code generators include in the messages generated static info about their own types and structure that can be later queried at compile time. This info comes in the form of a string describing the field messages and types in order. This can be parsed to achieve a pseudo-introspection[7] on runtime for any given message.

behavior_tree_ros relies on *ros_type_introspection* package to do the parsing of this static info. Later you can feed it a raw buffer containing a non-deserialized message, and it will construct a general tree (not a behavior tree) with the same structure as the original message, field names, types and their values.

This tree is not too user friendly, though. Figure 13 shows the tree representation of the *TwoNumbers* message defined back in section 3.3. Only the nodes' names are represented. Internally, each node contains an identifier of its type and its value.

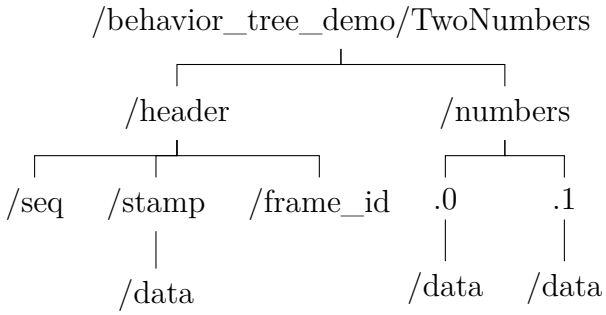


Figure 13: Tree representation of *TwoNumbers* message

ros_type_introspection offers a *flattened* version of the tree as well. A hierarchical structure is said to be flattened when it has only one level containing all the leaf nodes. Each leaf node is qualified by its full name, generated as the concatenation of all the nodes' identifiers that have to be transversed starting from the root to reach it. Figure 14 shows the flattened version of the tree, where actual values have been omitted.

```
{
  "behavior_tree_demo/TwoNumbers/header/seq":  ,
  "behavior_tree_demo/TwoNumbers/header/stamp/data":  ,
  "behavior_tree_demo/TwoNumbers/header/frame_id":  ,
  "behavior_tree_demo/TwoNumbers/numbers.0/data":  ,
  "behavior_tree_demo/TwoNumbers/numbers.1/data":
}
```

Figure 14: Flatten tree representation of *TwoNumbers* message

This structure is nearly identical to a flattened JSON, where its keys are JSON pointers as defined in RFC 6901. The only difference being that arrays indexes are preceded by a

[7] You actually need to get to know somehow what's the type of the message. Hence the need to state it in the plugin.

point instead of a slash. Replacing them by the correct delimiter enable us to instantiate a proper JSON object. Figure 15 shows the result.

```
{
  "header":
  {
    "seq": ...,
    "stamp": { "data": ... },
    "frame_id": ...
  },
  "numbers": [ { "data": ... }, { "data": ... } ]
}
```

Figure 15: JSON representation of *TwoNumbers* message

Now we have a common object for any type of message we may receive, with a proper hierarchy, type safety, iterators, etc. Leaf nodes in the behavior tree can be message agnostic and just operate on these structures.

There are, however, some catches to this approach. First, tree serialization is not well suited for messages containing big blobs of binary data, such as images or laser scans. If you are building a tree that uses this kind of messages, consider disabling serialization on your subscriber nodes and write more specialized nodes that operate using normal messages.

Also, while type safety is ensured during the process (meaning that floating numbers will be treated as floating numbers, strings as strings, unsigned as unsigned, etc), there are some granularity issues in the type information we have access to. The third party JSON library used internally only differentiates between unsigned, signed, decimal, integer or string (*ros_type_introspection* package does not have this problem). This means it's not possible to infer from the info provided in the JSON if a decimal variable is float or double, or if a number should be represented with 8, 16, 32 or 64 bits. To avoid any issues, decimals are casted to double, signed numbers to int64_t and unsigned to uint64_t when performing operations such as comparisons. This pretty much covers the whole process of receiving and manipulating messages.

The are some details not yet covered regarding the publication process, though. Publisher nodes use introspection to define ports with the same names and types as the fields in the original message. This way, users can just type values or use variables in *Groot*, and messages will be constructed and published automatically.

There's a constraint to this approach. Automatic publisher serialization is limited to messages containing only built-in types (string, integer and decimal numbers). This is not a limitation on the introspection method, but rather on the graphical interface. Given a message containing a non-fixed array of other sub-messages types, it's still not clear how to represent that easily in the GUI as ports so the user can input them. If you try to publish a message like that, you will get an error on runtime, stating that you should opt-out of the serialization by implement the functions *buildMessage* and *requiredMessageParameters*.

Let's see how to do it with an example. We are going to build a tree that publishes a non-fixed sized vector of *TwoNumbers* messages. Let's call this new message *TwoNumbersArray*. We first have to indicate that we want to opt-out of serialization when creating the publisher

node, by setting to false the second template argument of the publisher (see snippet 12 line 13).

Code Snippet 12: Disabling automatic publisher serialization

```
1 #include <std_msgs/String.h>
2 #include <behavior_tree_demo/TwoNumbers.h>
3 #include <behavior_tree_demo/TwoNumbersArray.h>
4
5 #include <behaviortree_cpp/bt_factory.h>
6 #include <behavior_tree_ros/behavior_tree_ros.hpp>
7
8 BT_REGISTER_NODES(factory)
9 {
10     using namespace BT_ROS;
11
12     factory.registerNodeType<PublisherNode<std_msgs::String>>("
          DemoPublishString");
13     factory.registerNodeType<PublisherNode<behavior_tree_demo::
          TwoNumbersArray, false>>("DemoPublishTwoNumbersArray");
14
15     factory.registerNodeType<SubscriberNode<behavior_tree_demo::
          TwoNumbers>>("DemoMonitorTwoNumbers");
16 }
```

Now we have to define what input info we need in order to build the message and how to actually build it by defining the functions *buildMessage* and *requiredMessageParameters*. Snippet 13 shows a possible implementation.

Code Snippet 13: Implementing explicit message generation

```
1 #include <behavior_tree_demo/TwoNumbersArray.h>
2
3 #include <behavior_tree_ros/behavior_tree_ros.hpp>
4
5 namespace BT_ROS
6 {
7     template <>
8     NodeParameters requiredMessageParameters<behavior_tree_demo::
          TwoNumbersArray>() { return { { "first",  "0.0"}, { "second",
          "0.0"}, { "third",  "0.0"}, { "fourth", "0.0"} }; }
9
10     template <>
11     behavior_tree_demo::TwoNumbersArray buildMessage(const ROSActionNode
          & _ros_node)
12     {
13         behavior_tree_demo::TwoNumbersArray array_message {};
14         array_message.array.resize(2);
15
16         array_message.array[0].numbers[0].data = _ros_node.getParam<
              float>("first").value();
17         array_message.array[0].numbers[1].data = _ros_node.getParam<
              float>("second").value();
18         array_message.array[1].numbers[0].data = _ros_node.getParam<
              float>("third").value();
19         array_message.array[1].numbers[1].data = _ros_node.getParam<
              float>("fourth").value();
20
```

```
21          array_message.array[0].header.stamp = ros::Time::now();
22          array_message.array[1].header.stamp = ros::Time::now();
23
24          return array_message;
25      }
26  }
```

The function *requiredMessageParameters* defines a map of pairs of strings. The first entry of each pair is the name of the port and the second entry the default value that *Groot* will display. In the example above we are generating a *TwoNumbersArray* message with two components and splitting each pair in its own variable (first, second, third and fourth).

The function *buildMessage* receives a *ROSActionNode* object as an argument. This object offers methods to access variables from the tree, such as *getParam*. The message is filled by searching the values of the port defined. Keep in mind that you can define these two functions in any file you want, but they should be compiled alongside the plugin and be placed in the *BT_ROS* namespace.

Figure 16 illustrates the use of this publisher. As it can be seen, the *TwoNumbersArray* node has the same ports defined in the *requiredMessageParameters* function. As it's the case with any other node, ports values can be either input directly in the text box or through variables in the tree.

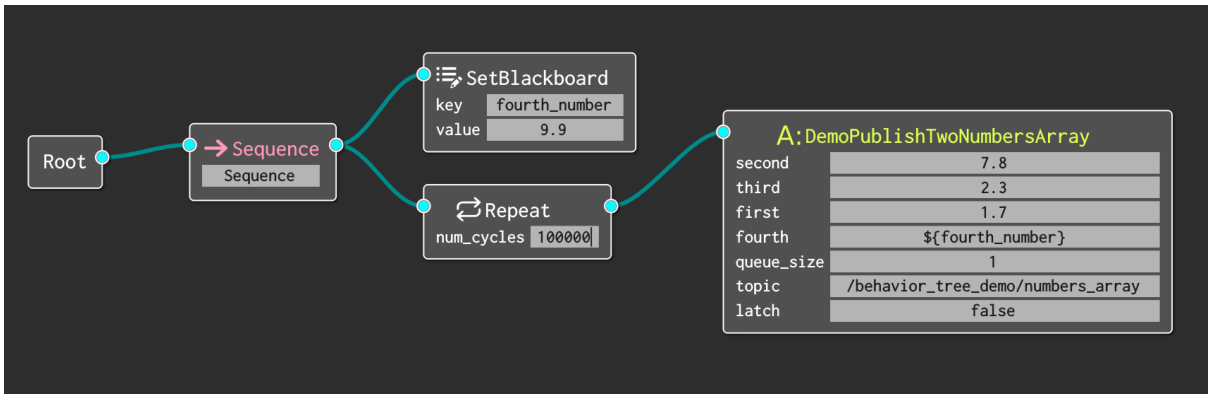


Figure 16: Tree implementing publisher with user-defined serialization.

You are free to build the message in any way you want. You can define as many ports as you need or no ports at all. You can also define just a single port that receives a message of the same type you want to publish. This is an easy way to create a "normal" publisher.